أكاديمية كاوست
KAUST ACADEMY

Advance Topics and Application

Dr. Fath U Min Ullah, Fellow of HEA
Lecturer in Artificial Intelligence, University of Lancashire,
Preston, United Kingdom

Course overview

أكاديمية كاوست
KAUST ACADEMY

- **Learning Objectives**
  - Introduce and familiarise you with Theory & Foundations of advanced topics
  - Learning about what is Generalization & Overfitting
  - How to use Dropout and Regularization concepts to avoid model overfitting.
  - You will be able to build a Deep Learning Model to adhere the above concepts
  - Understanding of real-time visualization of layers used in the model.

أكاديمية كاوست
KAUST ACADEMY

Contents

- Model Generalization
- Bias–variance tradeoff
- Regularization Techniques
- Dropout
- DIGIT Classification
- Visualization

أكاديمية كاوست
KAUST ACADEMY

Lecture Structure:

Local Time: 7:00 am BST

KSA Time: 9:00 am in Riyadh

Theorical Concepts:45 ~ Minutes

Implementation: 1 Hour

Visualization API tools: 5-10 minutes

Deep Learning Frameworks

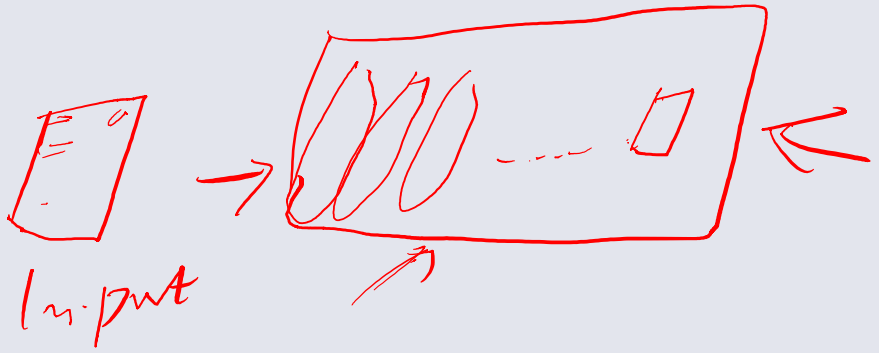
Deep learning frameworks are essential tools that simplify the process of designing, training, and deploying neural networks. Instead of implementing complex mathematical operations from scratch, these frameworks provide pre-built functions, optimized computation, and GPU acceleration to speed up development

1. **TensorFlow**: A highly scalable framework from Google.
2. **Keras** :A user-friendly API for deep learning.
3. **PyTorch** : A flexible and research-oriented framework from Facebook.
4. **Caffe:** A lightweight, fast framework for image processing.

Other Frameworks (MXNet, Theano, CNTK): Specialized frameworks with unique advantages

Model Generalization

Overfitting vs Underfitting



❑ **Overfitting**

- The model performs well on the training data but fails to generalize to new data.
- This usually happens when the model learns noise or irrelevant details.

Model Generalization

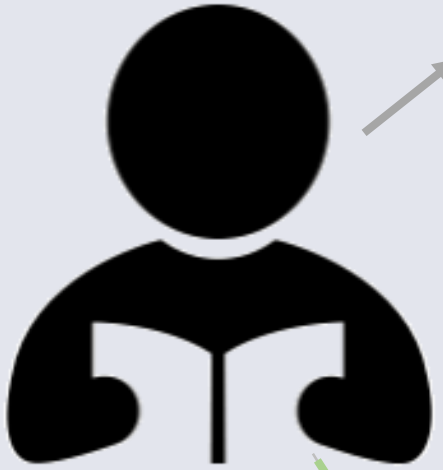
Overfitting vs Underfitting

❑ Underfitting

- The model is too simple and fails to capture the patterns in the data.
- Leading to poor performance even on the training data.

Model Generalization

A student studying for exam



- Memorizes exact questions answers from past exam
- Without truly understanding it.

Overfitting

Final exam
Same questions: Does great
Different questions (new data): will struggle

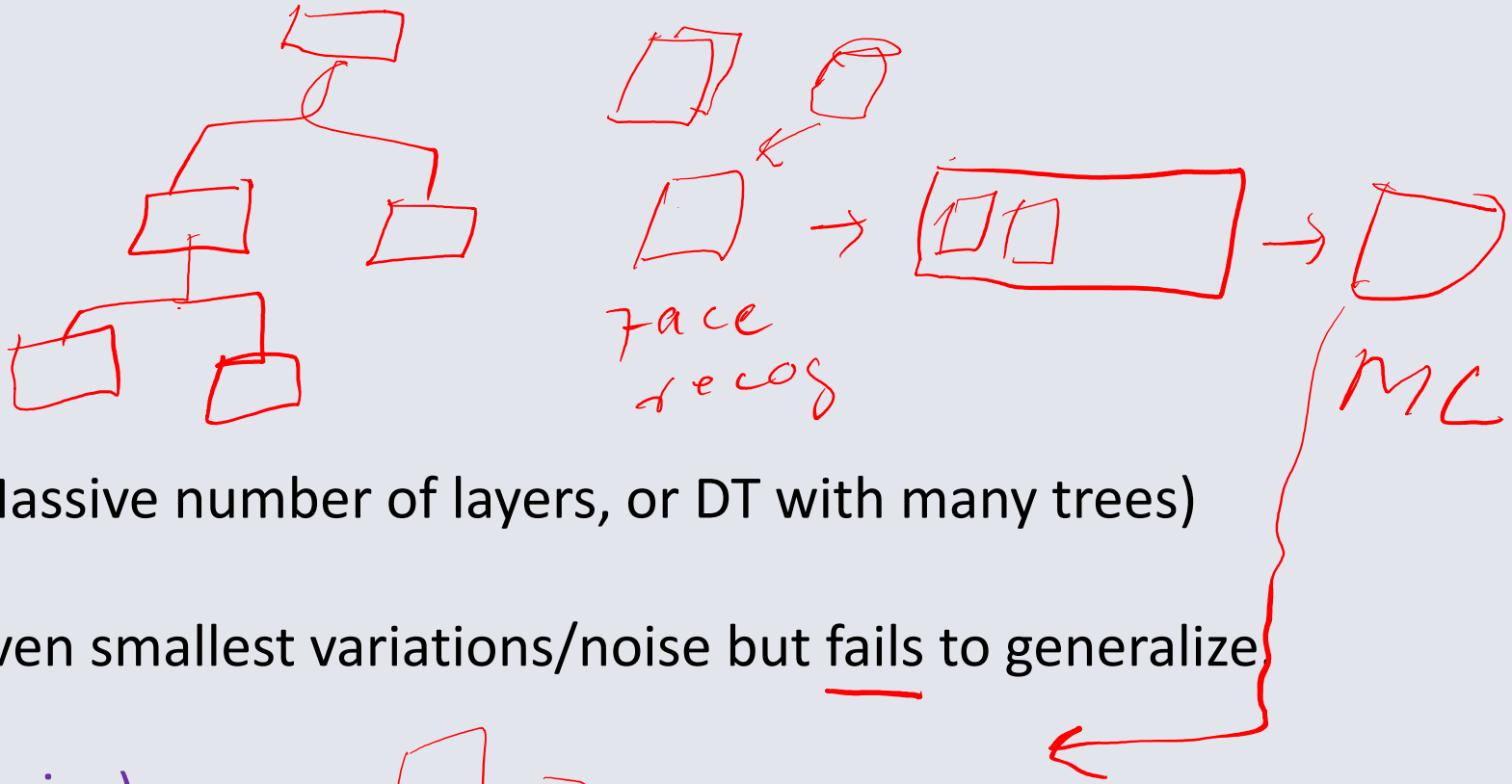
- Barely studies might only learn very basic concept

Underfitting

Final exam
Will struggle even if exam has same questions. Not enough to anwser

Model Generalization

In case of training a model



❑ **Overfitting**

- Model is too complex (Massive number of layers, or DT with many trees)
- Perfectly fit, capturing even smallest variations/noise but fails to generalize.

(Memorizing instead of learning)

Model Generalization

In case of training a model

❑ **Overfitting**

- **Training accuracy** is very high, but the **test accuracy** is much lower.
- A decision tree that splits the data repeatedly until each leaf node has only one data point (In CNN, too many filters/layers, no regularization)

Model Generalization

In case of training a model

❑ **Underfitting**

- Model is too simple (Very few layers, or DT with few trees)
- Happen if you use a linear model for a dataset with non-linear patterns is used

Model Generalization

In case of training a model

❑ **Underfitting**

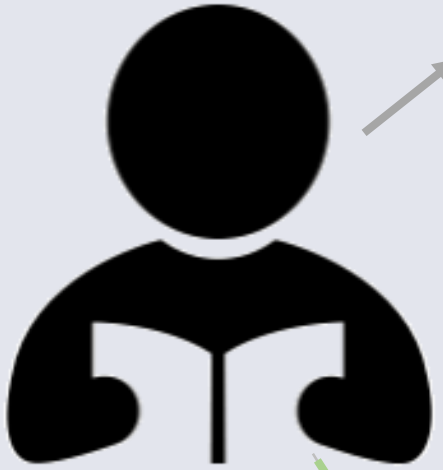
- **Training** and **testing** accuracy are low because the model fails to learn the relationships between the features and the target.

Model Generalization

Bias–Variance Tradeof

Good balance
- Student understands the material well
- Good on both homework and exam.

Two types of mistakes a student can make when learning



- Student memorizes the textbook word-for-word
- Perfect on homework

High bias (Overfitting)

Final exam
but fails on real exam

- Student is lazy and only learns very simple rules

High bias (Underfitting)

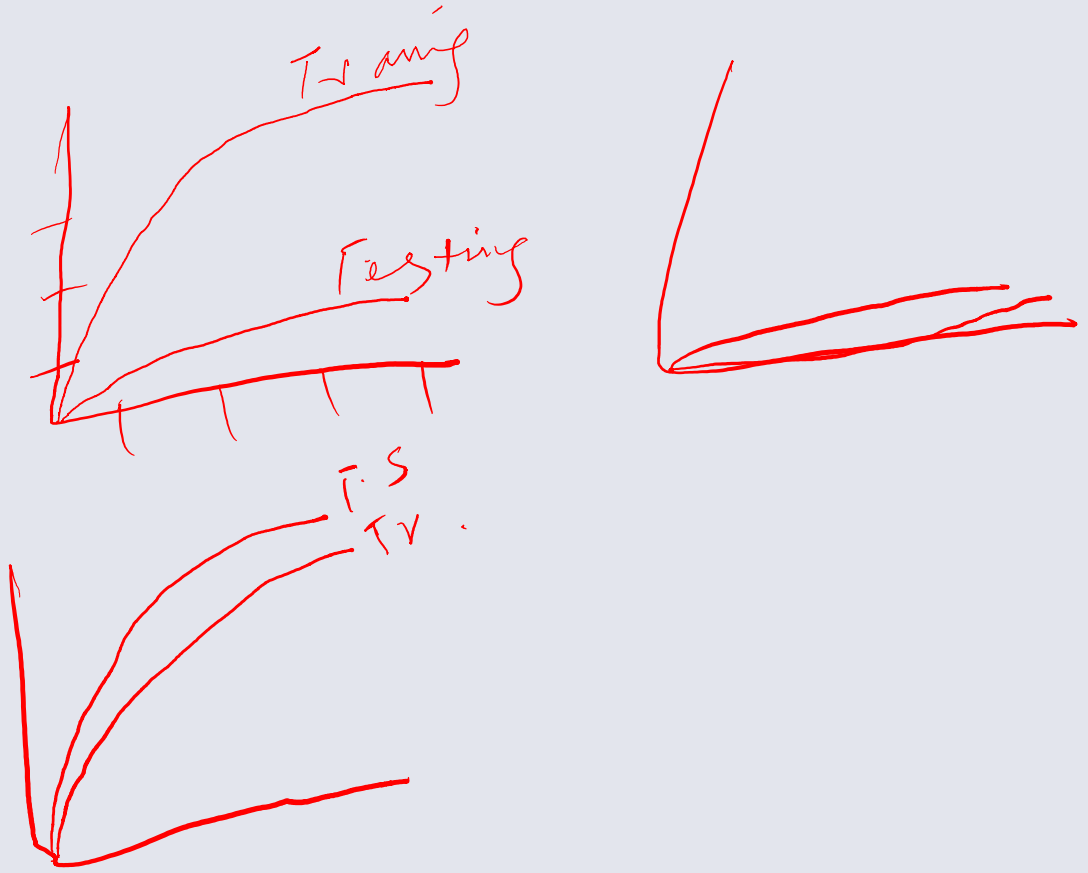
Always wrong

Model Generalization

Bias = error from too simple assumptions.

Variance = error from too much complexity.

balance between bias & variance.

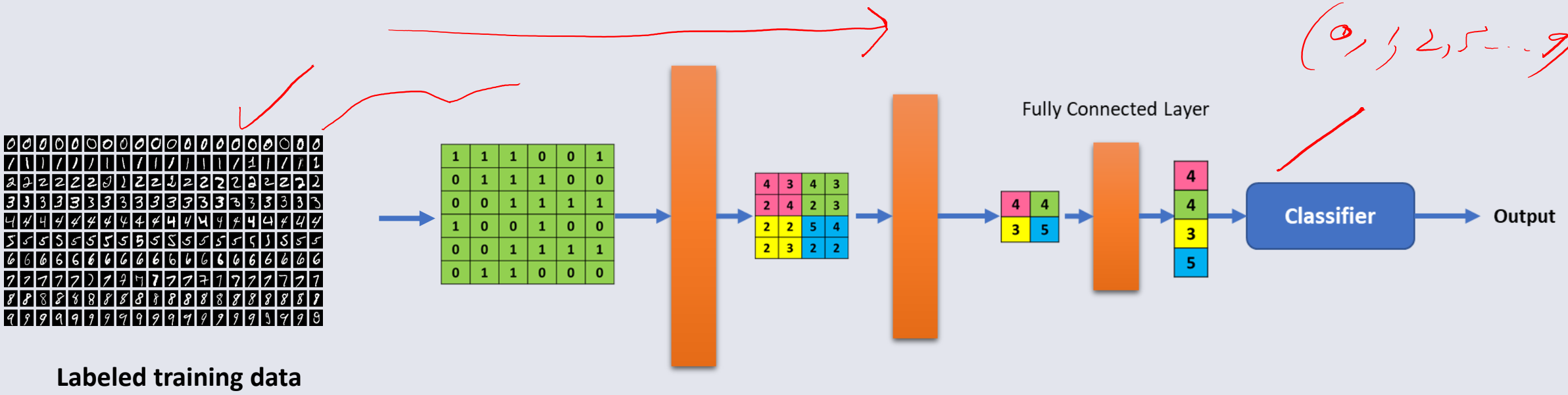


Model Generalization ???

Generalization: The ability of a model to perform well on **unseen data** (not just the training set).

Solving the above problems.

Model Generalization



Labeled training data

Regularization Techniques

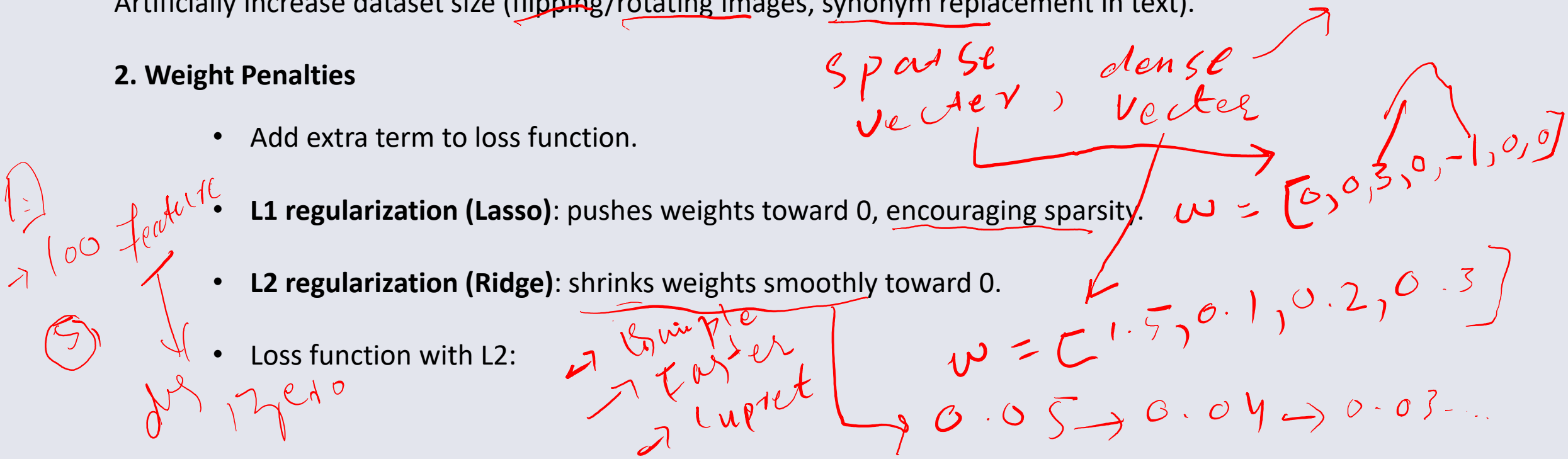
Regularization = I**mprove generalization and prevent overfitting**.

1. Data Augmentation

Artificially increase dataset size (flipping/rotating images, synonym replacement in text).

2. Weight Penalties

- Add extra term to loss function.
- **L1 regularization (Lasso)**: pushes weights toward 0, encouraging sparsity.
- **L2 regularization (Ridge)**: shrinks weights smoothly toward 0.
- Loss function with L2:



Regularization Techniques

epoch = 50

1 epoch = 7 mit

50 epoch

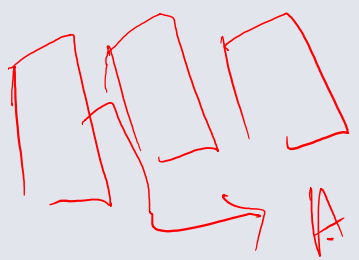
3. Early Stopping

Stop training when validation loss stops improving.Prevents overfitting by not letting the model learn noise.

±2,3 ↑↓

4. Batch Normalization

Normalizes intermediate activations, stabilizing training, acting as mild regularization.



Activ

large mean and variance

tiny [ ]

Dropout Layer

output = 0

h = [h1, h2, h3 ...]

Drop rate = 0.5

hi = 50% → 0

Proposed by Hinton (2014).

The Dropout Layer is a regularization technique used to prevent overfitting by randomly setting a fraction of input units to zero during training.

```
from tensorflow.keras.layers import Dropout
dropout_layer = Dropout(rate=0.5)  # Drops 50% of neurons randomly
```

It is not a standard part of the network or fully connected layer, but it is added to improve generalization by making the model more robust.

Dropout Layer

Proposed by Hinton (2014).

The Dropout Layer is a regularization technique used to prevent overfitting by randomly setting a fraction of input units to zero during training.

Digits Classification

MNIST = *Modified National Institute of Standards and Technology* dataset

70,000 images of handwritten digits (0–9)

60,000 training, 10,000 testing

Each image: **28 × 28 grayscale pixels**

Classic benchmark dataset in Machine Learning & Deep Learning

Digits Classification

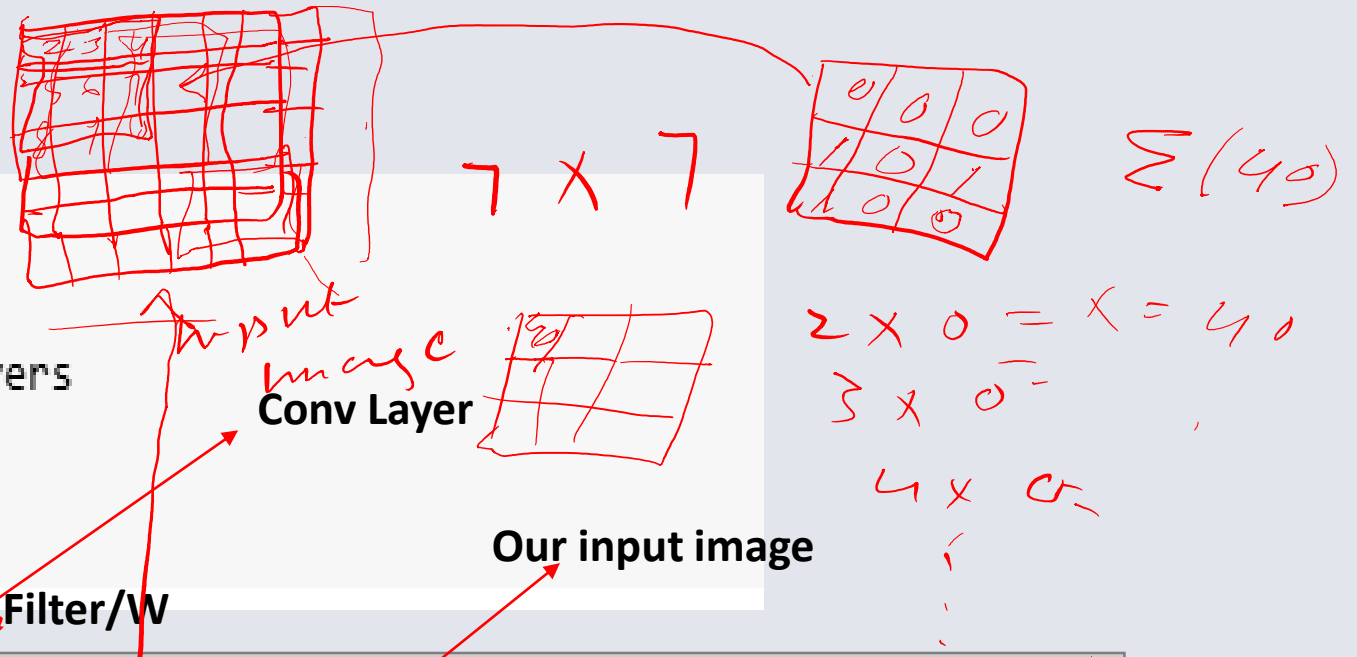
Input: Image of a handwritten digit

Output: Predicted class (0–9)

Type of problem: Multi-class classification

Digits Classification

```
[1]  1 import tensorflow as tf
     2 from tensorflow import keras
     3 from tensorflow.keras import layers
     4 import numpy as np
     5 import matplotlib.pyplot as plt
```




```
[3]  1 model = keras.Sequential([
     2     layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
     3     layers.MaxPooling2D((2,2)),
     4     layers.Conv2D(64, (3,3), activation='relu'),
     5     layers.MaxPooling2D((2,2)),
     6     layers.Flatten(),
     7     layers.Dense(64, activation='relu'),
     8     layers.Dense(10, activation='softmax')
     9 ])
```

Digits Classification

```
model = keras.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    # layers.Dropout(0.5),    # Drop 50% of neurons #Each forward pass uses
    layers.Dense(10, activation='softmax')
])

```

Convert to 1D

Detect edges/patterns

Reduce dimensions

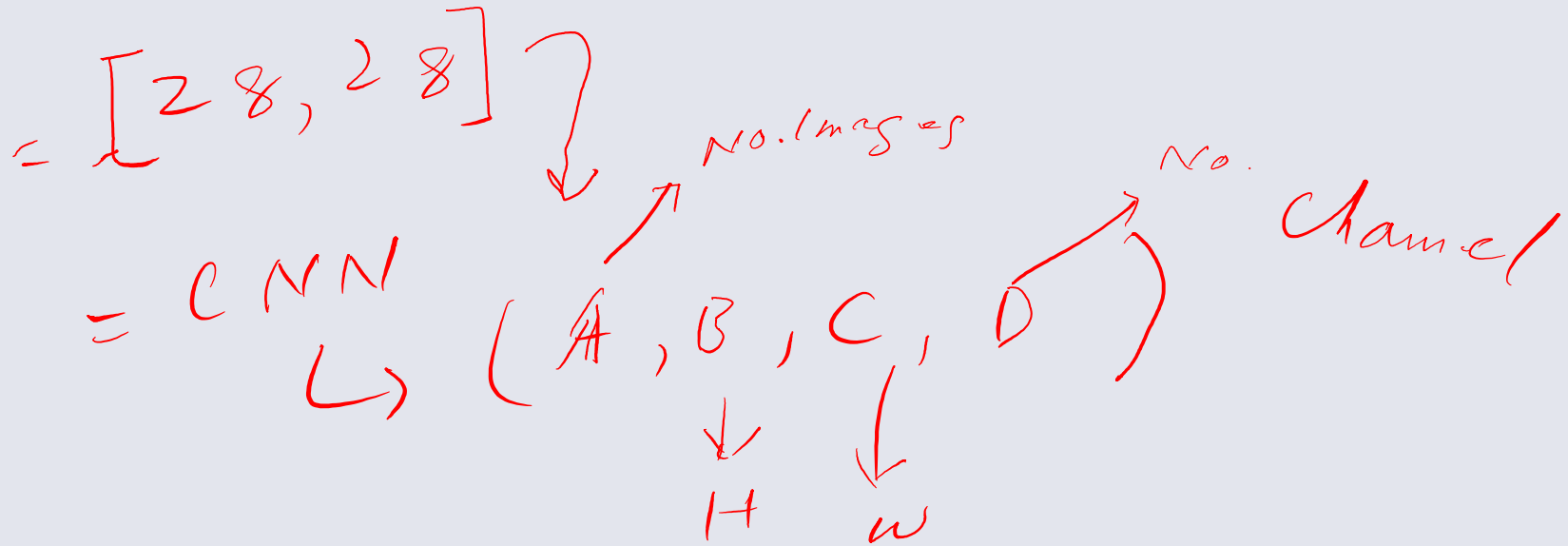
Digits Classification

```
10
11 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
[4]   1 history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

Digits Classification

3D Visualization of a Convolutional Neural Network


= [28, 28]
= CNN
(A, B, C, D)
No. Images
No. Channel
H
W

Discussion?